

DATA QUALITY IN DATABASE MIGRATIONS

Why and When to Use ICE SQLAlchemy Validation

Catching the data corruption that migration tools report as success

Hubert Dec · Patryk Dziki · Adrian Czapka · Dominik Dziedzic · Benjamin Sujka

The migration succeeded. The data did not.

Six documented incidents where no error fired — yet the data came out wrong.

The dangerous migration failure is the one that **does not** announce itself. No exception, no alert, the row count matches, and the tooling reports success. The corruption surfaces days or weeks later — through a customer, an analyst, or a regulator.

Organisation	What went wrong	Severity	Found by
Netflix Billing	Oracle NUMBER had no exact MySQL type — risk of silent truncation	MEDIUM	Pre-migration audit
Insurance (EU)	127 000 policies from the 1940s migrated as issued in 2040	MEDIUM	User, 3 weeks later
MySQL → Postgres	87 % of timestamps shifted by a whole timezone offset	HIGH	Customer report
OpenAI / Redis	Cache resequencing returned data to the wrong user sessions	HIGH	User reports
TSB Bank	1.3 billion records corrupted; customers saw strangers’ accounts	CRITICAL	Customers, 20 min later
TUI Aviation	Title “Miss” remapped to child mass — load sheet 1 244 kg light	CRITICAL	Post-flight audit

Every one passed its structural checks. The only failing check was the one nobody ran: **does this value still mean what it meant before?**

What every incident has in common

Different engines, different domains — one structural pattern.

1 • Validation confirmed movement, not meaning

Row counts matched, error logs were empty, connectivity checks passed. None of these verify that a value in the target carries the **same meaning** it had in the source. 87 % of timestamps were wrong while the migration reported a clean run.

3 • Testing covered the common case only

The insurance team validated date conversion on recent records and never reached the 1940s archive where the legacy two-digit-year bug lived. TUI tested typical passengers, not the “Miss” title against the weight calculation it actually fed.

2 • Errors were self-concealing

A corrupted timestamp still looks like a valid timestamp. A misdated policy is still a well-formed date. A 1 244 kg-light load sheet still parses as a normal load sheet. The corruption hides **inside** the normal appearance of the data — which is what makes it more dangerous than a crash.

4 • Detection came from users, not monitoring

TSB’s customers reported wrong accounts within 20 minutes of go-live. The timezone bug was reported by a customer, not an alert. OpenAI learned of the exposure from users. In no case did internal monitoring catch the corruption first.

The gap in standard migration tooling

Schema constraints and ORM validators guard new writes — not the rows already loaded.

Bulk loads — `pandas.to_sql`, ETL pipelines, `COPY`, hand-written SQL — write straight to the table and **bypass ORM validation entirely**. Millions of migrated rows can hold violations the model would reject on a fresh insert, and nothing in the normal toolchain re-checks them.

Mechanism	Catches a bad new INSERT?	Audits already-migrated rows?
<code>CheckConstraint</code> on the model	Often	Only if the DB enforces it and data is re-checked
ORM <code>#"@validates"</code> hook	On flush	Never — not triggered by bulk / raw SQL load
Row count vs. source	—	Volume only — says nothing about field correctness
<code>MigrationValidator</code> TOOL	—	Yes — scans every existing row with set-based SQL

Worse: with constraints disabled for load performance (a common bulk-import step), an engine will happily accept a `NULL` into a `NOT NULL` column. The violation simply **stays** in the table — discoverable only by an explicit, row-level audit.

What ICE SQLAlchemy Validation does

Declare the rule once, on the column. Run it against the data already in the database.

Rules live directly on the ORM column as `Annotated` markers — same file, same code review, no parallel config to drift out of sync. The engine reads those markers and runs set-based SQL against the live table, returning every row that breaks a rule.

```
from typing import Annotated
from sqlalchemy.orm import Mapped, mapped_column
from src.metadata import (
    Required, Unique, FloatMin, FloatMax,
    UniqueComposite, NoNaN,
)

class User(Base):
    __tablename__ = "users"
    id: Mapped[int]
    email: Mapped[Annotated[str, Required(), Unique()]]
    age: Mapped[Annotated[int, Required(),
                          FloatMin(0), FloatMax(120)]]
    country: Mapped[Annotated[str, Required(),
                              UniqueComposite("country", "city")]]
    city: Mapped[Annotated[str, Required()]]
    score: Mapped[Annotated[float, NoNaN()]]
```

Markers available in v1

Marker	Meaning
Required()	Value must not be NULL
Nullable()	Explicitly permits NULL
Unique()	Column values must be unique
UniqueComposite	Multi-column unique rule
FloatMin(n)	Numeric lower bound
FloatMax(n)	Numeric upper bound
NoNaN()	Reject IEEE NaN in a float

A non-optional `Mapped[T]` implies `Required()` automatically; `Mapped[T | None]` skips the null check. Legacy `Column(...)` with `Annotated` is supported too.

Which report failure modes does it catch?

Each v1 marker targets a specific top-10 pattern from the Sprint I report.

The library deliberately starts with the **structural** failure modes — the ones expressible as a per-row predicate. These are the patterns that bulk loads reintroduce most often and that field-level validation can verify with certainty.

Failure mode (report §3)	Real incident	Severity	Marker that catches it
Numeric precision / out-of-range value	Netflix billing truncation	MEDIUM	FloatMin / FloatMax
Silent NULL after a bulk load	Insurance nulls, TSB FK gaps	CRITICAL	Required()
Duplicate records from re-runs	Non-idempotent ETL pipelines	LOW	Unique()
NaN written into a float column	Pandas / ETL artefacts	MEDIUM	NoNaN()
Composite key uniqueness broken	Multi-tenant migrations	HIGH	UniqueComposite

Not yet in v1 (future work): timezone / semantic drift, category remapping, encoding corruption, and cross-table referential integrity. Each needs a domain-specific or multi-table check — and the `parser.py` pipeline already accepts custom markers, so they slot in without engine changes.

How the validator runs

SQL-first, streaming, two passes — no ORM overhead and no full table held in memory.

Pass 1 — Field rules

parser.py reads the Annotated metadata of every mapped field and builds one validator per marker. Each emits a single SQL SELECT targeting **only** the violating rows (e.g. WHERE age < 0), streamed with yield_per=5000 for constant memory use at any scale.

Pass 2 — Duplicate rules

All Unique() and UniqueComposite targets are gathered, then a GROUP BY ... HAVING COUNT(*) > 1 query finds every duplicate key set in one round-trip per constraint.

Python only where SQL can't

NoNaN() pre-filters non-null rows in SQL, then checks math.isnan() in Python — because NaN comparison behaviour is not portable across SQLite versions.

Run it — three lines

```
from sqlalchemy.orm import Session
from src.engine import MigrationValidator
from src.pandas_ import (
    errors_to_dataframe, summarize_errors,
)

with Session(engine) as session:
    errors = MigrationValidator(session) \
        .validate(User)

report = errors_to_dataframe(errors)
summary = summarize_errors(report)
```

Example summary output

kind	column	count
null	email	52
duplicate	email	50
float_min	age	20

Gate your migration pipeline with pytest

One assertion turns the whole audit into a pass/fail CI check.

The library registers a pytest plugin through `pyproject.toml`, exposing a `migration_validator` fixture and an `assert_table_valid` helper. Point it at each model and the build fails the moment any row is dirty — no separate reporting step to remember.

```
# test_migration.py
from src.pytest_validation_plugin import (
    assert_table_valid,
)

def test_users_after_migration(session):
    assert_table_valid(session, User)

def test_policies_after_migration(session):
    assert_table_valid(session, Policy)
```

On failure the assertion prints the total error count and the summary DataFrame — identical to the pandas report.

Plugin entry point

```
[project.entry-points.pytest11]
ice_sqlalchemy_validation = \
    "src.pytest_validation_plugin"
```

Add it to CI when...

- staging data mirrors production volume
- an ETL / bulk load ran outside the ORM
- the schema changed since the last migration
- a hotfix touched legacy rows
- cut-over needs a sign-off gate

A red test here costs an engineer an hour. The same defect post-go-live cost TSB **£330 million**.

When to run validation in a project

Not a one-off — a gate that fires every time data moves.

Pipeline stage	Trigger	What to validate
After a bulk load to staging	Every ETL / <code>to_sql</code> run	All columns that carry markers
Before UAT begins	Manual gate or CI check	Every model; review the summary DataFrame
After a hotfix on legacy data	On merge to the staging branch	Only the affected models
Before production cut-over	Required sign-off step	Full schema; diff summary against a baseline
Just after production cut-over	Smoke test in the first 30 min	Critical tables; alert on any error

The one principle

Validate at the **volume and shape** of production data — never a 100-row sample. TSB’s test environment never reached production scale, and that is exactly where its edge cases lived.

Cheap to wire in

Because the check is just SQL over an existing table, running it on staging adds seconds, not a new pipeline. The same models power the demo in `examples/validate_dirty_database.py`.

Summary

Structural integrity today; semantic checks tomorrow — always declared on the model.

Why it exists

- Real incidents prove a “successful” migration can still corrupt data
- Row counts and clean logs never verify field-level correctness
- Bulk loads bypass ORM validation, so violations survive undetected
- Users find the problem before monitoring ever does

How to adopt it

1. Annotate mapped columns with markers
2. Run `MigrationValidator(session).validate(Model)`
3. Review `errors_to_dataframe / summarize_errors`
4. Add `assert_table_valid` to the staging suite, gate cut-over on a clean run

What it covers now

- NULL violations — `Required()`
- Numeric out-of-range — `FloatMin`, `FloatMax`
- Uniqueness — `Unique`, `UniqueComposite`
- IEEE NaN in floats — `NoNaN()`

Where it’s going

- Category / ENUM remapping validators
- Cross-table referential integrity checks
- Timezone-aware timestamp comparison
- Custom domain markers via the existing `parser.py` pipeline

```
uv run pytest · uv run python examples/validate_dirty_database.py
```